

中国矿业大学计算机学院

2023 级本科生课程实验报告

课程名称:	大数据架构技术课程设计
报告时间:	2025.12.7
学生姓名:	王志豪
学 号:	08232337
专业班级:	数据科学与大数据技术 1 班
任课教师:	雷小锋

成绩考核

编号	课程教学目标	占比	得分
1	目标 1:掌握 Linux 操作系统基本的 Shell 命令。	20%	
2	目标 2: 掌握 Hadoop 的安装部署和开发。	20%	
3	目标 3: 掌握 Spark 的安装部署和开发。	20%	
4	目标 4: 掌握 Flink 的安装部署和开发。	20%	
5	目标 5: 掌握较大文档的编辑排版方式，做到文档形式优雅，内容充实真诚。	20%	
总成绩			
指导教师		评阅日期	

目录

实验一 Linux 环境实践.....	1
1. 实验基本信息	1
2. 实验目标和实验环境	1
3. 实验内容	1
4. 实验步骤和结果	1
4.1 使用联机帮助命令，包括 man 命令和 info 命令	1
4.2 使用目录和文件操作命令	5
4.3 Linux 用户和组	8
4.4 使用 vim 编辑器	11
4.5 yum 命令安装软件	12
4.6.集群节点配置、通信与免密互信 。	13
4.7.配置环境变量与安装 Java 开发环境	15
4.8. ssh 客户端远程操作虚拟机。	15
4.9. Docker&Kubernetes 安装部署	16
5. 结论与讨论	16
实验二 Hadoop 安装部署与开发	16
1. 实验基本信息	16
2. 实验目标和实验环境	17
3. 实验内容	17
4. 实验步骤和结果	17
4.1. 单机本地部署 Hadoop	17
4.2. 伪分布部署 Hadoop	18
4.3. 使用 HDFS 命令	19
4.4. 分布式部署 Hadoop	20
4.5. MapReduce 编程实例：词频统计	23
4.6. MapReduce 编程实例：排序	24
5. 结论与讨论	24
实验三 Spark 安装部署与开发	25
1. 实验基本信息	25
2. 实验目标和实验环境	25
3. 实验内容	25
4. 实验步骤和结果	26
4.1 Spark 的 Local 部署	26
4.2 Spark 的 Standalone 部署	27
4.3 Spark 的 Yarn 部署	28
4.4 Spark shell 与 RDD 基本操作	30
4.5 Spark shell 编程实例	30
4.6 使用 SBT 构建工具进行 Spark 应用程序开发	35
4.7 使用 Maven 构建工具进行 Spark 应用程序开发	35

5. 结论与讨论.....	36
实验四 Flink 安装部署与开发	37
1. 实验基本信息	37
2. 实验目标和实验环境	38
3. 实验内容	38
4. 实验步骤和结果	38
4.1 Local 模式安装部署 Flink	38
4.2 Standalone 模式安装部署 Flink	40
4.3 Standalone 模式安装部署 Flink	41
4.4 YARN 模式安装部署 Flink	45
4.5 Flink 应用程序开发	46
5. 结论与讨论.....	48

实验一 Linux 环境实践

1. 实验基本信息

实验名称：熟悉 Linux 操作系统的常用命令

实验日期：2025.9.15

实验地点：计算机学院楼计-19 机房

2. 实验目标和实验环境

实验目标：熟悉 Linux 操作系统的常用 Shell 命令

实验环境：MobaXterm、CentOS7

3. 实验内容

1.使用联机帮助命令，包括 man 命令和 info 命令; 2.使用目录和文件操作命令; 3.Linux 用户和组; 4.使用 vim; 5.yum 命令安装软件; 6.集群节点配置、通信与免密互信; 7.配置环境变量与安装 Java 开发环境; 8. ssh 客户端远程操作虚拟机。

4. 实验步骤和结果

4.1 使用联机帮助命令，包括 man 命令和 info 命令

对于 man、whatis、info、apropos、--help，虽然都是查看命令手册的命令，但是也有所区别，例如 man 是用来查看详细手册页

的，whatis 是查看简短描述的，info 是用来查看 GNU 软件 Info 文档的，apropos 是用来根据关键字搜索相关命令及说明的，而—help 则是查看快速指南的。如下是我经过学习手册后测试各个 shell 命令的结果和分析。

ls 命令：确定当前目录下的文件名与相关属性

```
[root@centos7-070 ~]# ls
bin  dev  home  lib64  media  opt  root  sbin  srv  tmp  var
boot  etc  lib  lost+found  mnt  proc  run  selinux  sys  usr
```

图 1 ls 命令结果

使用该命令后可以快速查看当前文件目录下有哪些文件，当然也可以指定文件目录去查询，比如可以用 ls /usr，这样查询的就是 /usr 目录下面的文件。

cd 命令：切换目录命令，如下切换到/文件路径下

```
[root@centos7-070 ~]# cd /
[root@centos7-070 /]# ls
```

图 2 cd 命令结果

基础的用法就是 cd [目录名称]，即可切换到相应的目录下。

echo 命令：相当于 C 语言中的 printf，在 shell 中，可以打印变量的值，或者输出指定的字符串，可以将结果写入到文件，也可以打印在终端。

字符串输出：echo 'hello world'，即可打印出相应字符串。

```
[root@centos7-070 ~]# echo 'hello world'
hello world
```

图 3 echo 命令结果

写入 hello.txt 文件：

```
[root@centos7-070 /]# vi hello.txt
[root@centos7-070 /]# echo "hello world" > hello.txt
```

图 4 写入 hello.txt

type 命令：可以显示指定命令的信息，判断给出的指令是内部命令、外部命令（文件）、别名、函数、保留字或者不存在（找不到），查看 shell 内置命令：测试不同的参数的运行结果如图 5。

```
[root@centos7-070 /]# type -a echo
echo is a shell builtin
echo is /usr/bin/echo
[root@centos7-070 /]# type -p echo
[root@centos7-070 /]# type -t echo
builtin
[root@centos7-070 /]# type echo
echo is a shell builtin
```

图 5 type 不同参数命令结果

通过 cd 和 ls 命令浏览并熟悉 Linux 文件系统结构：

```
[root@centos7-070 /]# ls
bin    dev    hello.txt  lib    lost+found  mnt    proc    run    selinux  sys  usr
boot   etc    home       lib64  media       opt    root    sbin   srv      tmp  var
```

图 6 查看 Linux 文件系统

在 Linux 的/目录下使用 ls，查看 Linux 的文件系统结构如图 6 所示，Linux 的文件系统采用单一根目录结构，以根目录“/”作为整个系统层级的起点，所有文件和设备都以目录树的方式从这里向下扩展。系统将不同功能的内容分门别类，例如 /bin 和 /sbin 存放基本的可执行程序，/etc 存放系统配置文件，/usr 保存大多数用户应用及其库文件，/var 存放经常变化的数据如日志和缓存，/home 为普通用户提供个人目录，而设备文件以特殊方式呈现在 /dev 下。无论是真实磁盘、虚拟文件系统还是外接设备，最终都被挂载到这棵目录树的某个节点，形成统一、清晰且高度抽象的文件组织

体系。通过学习 Linux 的基本文件结构，我逐渐理解文件、目录和路径的概念，理解 Home 路径、绝对、相对路径的区别。

A. 理解文件、目录和路径的概念

文件：计算机系统中用于存储信息（数据）的基本单位。

目录：文件系统中的特殊文件，它的主要功能是存储一组文件和其他目录，Linux 中的文件目录是以单根的方式组织文件的。

路径：是一个字符串，它精确地描述了文件或目录在文件系统树形结构中的位置

B. 理解 Home 目录、绝对路径和相对路径（~ / ./ ..）

Home 目录：存储用户的目录，用户的主目录，在 Linux 中，每个用户都有一个自己的目录，一般该目录名是以用户的账号命名的

绝对路径：就是以当前目录为起点出发的路径

相对路径：就是从根目录出发的路径

C. 绝对路径和相对路径的使用

cd ~：若不指定用户名称默认切换到当前用户的家目录

cd -：返回到上次执行 cd 命令的目录，即上次切换之前所在的目录。

cd..：返回父目录，即上一层目录

相对路径使用：在/目录下用 cd 进入/usr，可以直接 cd usr，也

可以 `cd ./usr`

绝对路径使用：要输入完整的路径名称

感受与体会：这一板块主要以学习命令为主，前面的命令测试部分其实并不难，最后的绝对路径和相对路径这一块起初让我有点困惑，因为我不太分得清 `./` 和 `/` 的区别，后来自己反复测试后才明白 `./` 是当前目录下的相对路径的意思，而 `/` 在 Linux 里是一个单独的目录名。

4.2 使用目录和文件操作命令

pwd：显示当前工作目录的绝对路径。例如我在 `/` 路径,输入 `pwd` 即可得到返回 `"/"`。

mkdir：创建新的目录。使用 `mkdir newfile`，创建一个 `newfile`

```
[root@centos7-070 ~]# mkdir newfile
[root@centos7-070 ~]# ls
bin  dev  hello.txt  lib  lost+found  mnt  opt  root  sbin  srv  tmp  var
boot  etc  home  lib64  media  newfile  proc  run  selinux  sys  usr
```

图 7 mkdir 运行结果

mv：移动文件或目录。使用 `mv hello.txt newfile`，将 `hello.txt` 移动到 `newfile`。

```
[root@centos7-070 ~]# ls
bin  dev  hello.txt  lib  lost+found  mnt  opt  root  sbin  srv  tmp  var
boot  etc  home  lib64  media  newfile  proc  run  selinux  sys  usr
[root@centos7-070 ~]# mv hello.txt newfile
[root@centos7-070 ~]# ls /newfile
hello.txt
```

图 8 mv 运行结果

rmdir：删除空目录。假设在 `/` 目录下有一 `empty` 空文件夹，现在我使用 `rmdir empty`，去删除它，发现是可以删除的，但是假如 `empty` 下有文件，现在再用这个命令则无法实现删除功能，如图 9

所示。

```
[root@centos7-070 ~]# ls /empty
hello.txt
[root@centos7-070 ~]# rm empty
rm: cannot remove 'empty': Is a directory
```

图 9 rmdir 运行结果

rm: 删除目录或文件，先删除 newfile 下的 hello.txt 文件。

```
[root@centos7-070 ~]# rm /newfile/hello.txt
rm: remove regular file '/newfile/hello.txt'? y
[root@centos7-070 ~]# ls /newfile
[root@centos7-070 ~]#
```

图 10 rm 运行结果

touch: 修改文件或目录的时间属性/创建空文件。这里我们 touch 一个 file，输入 touch file 命令，不存在这个目录就会先创建这个目录。检查该目录下是否存在 file 这个目录，发现是存在的：

```
drwxr-xr-x  2 root  root  4096 Sep 15 07:30 file
```

图 11 touch 运行结果 1

而后我们测试是否会修改 file 的时间属性，我们再次 touch file，检查时间属性是否被修改，发现确实被修改：

```
drwxr-xr-x  2 root  root  4096 Sep 15 07:31 file
```

图 12 touch 运行结果 2

cp: 拷贝文件或文件夹。将文件 hello.txt 的内容复制给 hey.txt，hey 文件不存在则会创建它：cp hello.txt hey.txt。

cat: 显示文件内容。

```
[root@centos7-070 ~]# cat hello.txt
hello world!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

图 13 cat 运行结果

more: 用于分页显示文件内容，当文件内容较长无法在一屏中完整展示时，它会按页输出，并在每页底部等待用户按键继续。在测试的过程中，我发现它的交互功能很有限，只能单向向下浏览文件内容。

less 支持前后翻页、搜索、跳转、滚动等多种导航方式，并且不会一次性将整个文件读入内存，因而适用于大文件。测试的总体感受是强大、流畅，几乎没有明显缺点，只是需要一定熟悉时间。

head 用于查看文件开头的若干行，默认显示前 10 行，适用于快速了解文件格式、结构或开头部分内容。测试的整体体验是快速、直接，但只能提供很有限的视图。

tail 用于查看文件结尾的内容，默认返回最后 10 行，同样可以通过 `-n` 自定义输出行数。整体感受是非常实用，尤其适合调试日志，但需要注意避免错误使用造成窗口“卡住”的错觉。

tar: 文件归档和解档，将 `hello.txt` 和 `hey.txt` 文件归档 `learn.tar` 文件中并保存到当前目录，如图 14 所示

```
[root@centos7-070 /]# tar -cvf learn.tar hello.txt hey.txt
hello.txt
hey.txt
[root@centos7-070 /]# ls
15  boot  file  home  lib64  mnt  proc  sbin  srv  usr
15:26 dev  hello.txt  learn.tar  lost+found  newfile  root  selinux  sys  var
bin  etc  hey.txt  lib  media  opt  run  Sep  tmp
```

图 14 归档结果

将 `learn.tar` 解压缩到 `file` 目录下：

```
[root@centos7-070 /]# tar -xvf learn.tar -C file
hello.txt
hey.txt
[root@centos7-070 /]# ls /file
hello.txt  hey.txt
```

图 15 解压结果

“*”的使用：匹配 0 或多个字符，这里我们在 less 环境中去测试，搜索 hey.txt 文件中的 g 开头 a 结尾的字符串，在 less 搜索环境下输入/g*a，发现搜索的结果并不是我们预期的那样，以 g 开头 a 结尾的单词，在搜索有关资料后发现，less 搜索使用的是正则表达式，而我们 g*a 的实际意思是：匹配任意数量（包括 0 次）的 g + 一个 a，如图 16 左图所示，如果我们要实现正常的功能，要输入/^g.*a\$命令，才能达到我们的预期，如图 16 右图所示。

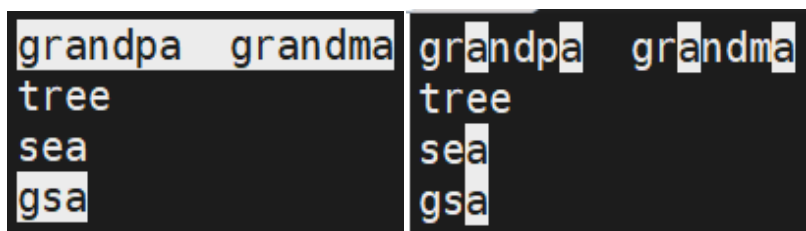


图 16 less 中*通配符使用结果

“?”的使用：匹配任意一个字符，这里我们搜索以 g 开头 a 结尾的文件名，中间只隔一个字符的字符串，如图 17 所示，可以发现只有 g_a 和 gba 匹配成功；当然如果在 less 中使用相同功能的通配符，则需要用“.”代替“?”。

```
[root@centos7-070 empty]# ls
ab.txt g123a.txt g_a.txt ga.txt gba.txt g.txt gxa.log hello.txt hey.txt
[root@centos7-070 empty]# ls g?a.txt
g_a.txt gba.txt
```

图 17 ? 通配符使用结果

4.3 Linux 用户和组

useradd: 创建新用户，这里我们输入 useradd wzh，而后用 cat /etc/passwd 去查看是否存在我们刚刚添加的用户，发现是存在的，如图 18 所示。

```
[root@centos7-070 ~]# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
adm:x:3:4:adm:/var/adm:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
operator:x:11:0:operator:/root:/sbin/nologin
games:x:12:100:games:/usr/games:/sbin/nologin
ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
nobody:x:99:99:Nobody:/:/sbin/nologin
systemd-network:x:192:192:systemd Network Management:/:/sbin/nologin
dbus:x:81:81:System message bus:/:/sbin/nologin
sshd:x:74:74:Privilege-separated SSH:/var/empty/ssh:/sbin/nologin
wzh:x:1000:1000::/home/wzh:/bin/bash
```

图 18 useradd 运行结果

userdel: 删除用户，这里输入 `userdel wzh`，再去使用 `cat /etc/passwd` 去查看 `wzh` 账户是否被删除，发现我们成功删除。

who: 查看当前登录用户，输入 `who` 即可，可以发现我们当前的账户是 `root`，并且可以看到 `ip` 地址、登录时间等一系列信息。

```
[root@centos7-070 ~]# who
root      pts/3      2025-09-15 06:13 (10.3.171.195)
```

图 19 who 运行结果

whoami: 显示使用者的用户名，它仅仅显示当前用户的用户名，没有其他的例如 `ip` 地址类的信息，我们当前的账户是 `root`。

passwd: 设置用户的密码，输入 `passwd [用户名]`，系统会弹出新的密码设置行，输入新的 `password` 就可以改密码了。如图 20

```
[root@centos7-070 ~]# passwd root
Changing password for user root.
New password:
BAD PASSWORD: The password is a palindrome
Retype new password:
passwd: all authentication tokens updated successfully.
```

图 20 修改密码结果

su: 切换当前用户，这里我们 `su wzh`，系统即切换到 `wzh`。

groupadd: 创建用户组，这里输入 `groupadd newgroup`，而后

输入 `cat /etc/group` 查询是否存在这个组，查询的内容是 `new-group:x:1001:`

groupdel: 删除用户组，这里我们删除 `newgroup`，输入 `cat /etc/group` 去查询的时候发现该组已经被删除。

groups: 显示用户所属的用户组，使用方法就是输入 `groups` 即可，这里我们查询 `root` 所属用户组，我的 `root` 属于 `root` 组。

cat /etc/passwd: 查看所有用户，该功能已经在 `useradd` 中试验过，因此不再赘述。

cat /etc/group: 查看所有用户组，该功能已经在 `groupadd` 中试验过，因此不再赘述。

chown: 改变文件属主。这里我们将 `hello.txt` 划分给 `wzh` 用户，输入 `chown wzh hello.txt`，而后去查看，发现 `hello.txt` 已经属于 `wzh` 组中的 `wzh` 用户，`ls -l` 的信息是：`-rw-r--r-- 1 wzh wzh 40 Sep 15 07:45 hello.txt`。

chmod: 修改文件权限。这里将目录及其子目录下所有文件的读写权限设置为所有用户可读写，但不可执行，也就是 `chmod 666 hello.txt`，最后我们查看 `ls -l` 发现权限已经改为 `rw-rw-rw-`，设置完毕，如图 21 所示。



```
-rwxrw-rw- 1 root root 0 Dec 7 07:13 hello.txt
```

图 21 修改权限结果

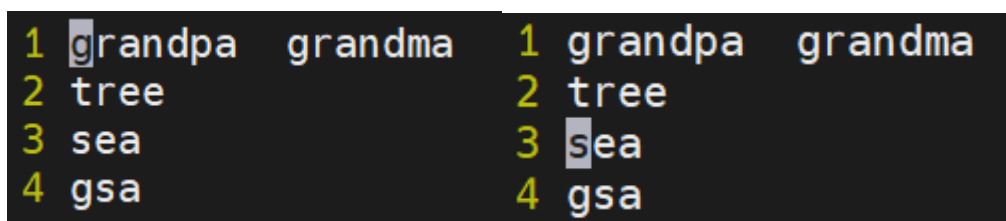
4.4 使用 vim 编辑器

首先是**命令模式**，我们在进入 vim 编辑器后会发现它和我们平常的文本输入方式不同，一开始我不知道怎么输入，以为是键盘坏了，其实这时 vi 编辑器处于命令模式，键盘的按键输入是作为编辑器的指令来使用的。在本次测试中，我分别测试了四种命令：**移动光标、删除文本、复制粘贴替换、Undo 与 Redo。**

在完成这些命令的测试时，我遇到了一些问题，比如在我要使用 shift+g 将光标移动到文件尾时，怎么按都没有用，最后发现是因为我开了大写，大写的 G 是不能完成这个命令的操作的，也就是说 vim 编辑器中的命令是严格区分大小写的，这与我平常使用的编辑器的快捷操作完全不一样。

而后是**输入模式**，在命令模式下按下 i 就进入了输入模式，由于和一般的编辑器功能差不多，因此就不再过多赘述了。

最后是**底行命令模式**在命令模式下，输入 ":" 即可进入底行命令模式，底行命令模式可以输入单个或多个字符的命令，可用的命令非常多，我们在这里测试了：:w filename（将文本保存为文件 filename）、wq（保存并退出 vim）、q（退出 vim）、q!（强制退出 vim）、set nu（显示行号）、/[查找的关键字]（查找关键字）。命令都不难，我们这里仅展示使用 set nu 和查找关键字的功能。



```
1 grandpa grandma 1 grandpa grandma
2 tree              2 tree
3 sea               3 sea
4 gsa               4 gsa
```

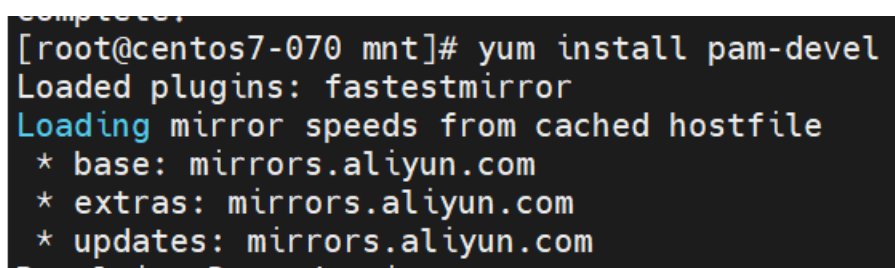
图 22 set nu 运行结果

如图 22 左图所示，这时输入 set nu 后出现了行号，而右图则表示在输入/sea 时，光标会定位到 sea 的开头。

4.5 yum 命令安装软件

yum 的安装过程分为三步：1.先对原 repo 文件进行备份；2. 下载对应版本 repo 文件，放入 /etc/yum.repos.d/；3. 生成缓存

安装完成后我们可以测试安装 pam-devel，输入 yum install pam-devel 即可进行安装，如图 23 所示。



```
complete.
[root@centos7-070 mnt]# yum install pam-devel
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
* base: mirrors.aliyun.com
* extras: mirrors.aliyun.com
* updates: mirrors.aliyun.com
```

图 23 安装 pam-devel

利用 yum 的功能，还可以找出以 pam 为开头的软件名称有哪些？输入 yum list pam*，利用通配符*，如图 24 所示。


```

[root@centos7-070 mnt]# yum list pam*
Loaded plugins: fastestmirror
Loading mirror speeds from cached hostfile
 * base: mirrors.aliyun.com
 * extras: mirrors.aliyun.com
 * updates: mirrors.aliyun.com
Installed Packages
pam.x86_64                                1.1.8-23.el7                                @base
Available Packages
pam.i686                                  1.1.8-23.el7                                base
pam-devel.i686                            1.1.8-23.el7                                base
pam-devel.x86_64                          1.1.8-23.el7                                base
pam_krb5.i686                             2.4.8-6.el7                                 base
pam_krb5.x86_64                           2.4.8-6.el7                                 base
pam_pkcs11.i686                           0.6.2-30.el7                                base
pam_pkcs11.x86_64                         0.6.2-30.el7                                base
pam_snapper.i686                          0.2.8-4.el7                                 base
pam_snapper.x86_64                        0.2.8-4.el7                                 base
pam_ssh_agent_auth.i686                   0.10.3-2.23.el7_9                          updates
pam_ssh_agent_auth.x86_64                 0.10.3-2.23.el7_9                          updates

```

图 24 找出 pam 开头的软件呢名称

当然，我在安装的时候出现了一些问题，比如缺少 repodata 的问题，这个问题根据教程重新生成好文件后也得以解决；还有就是由于 centos7 已经在去年 EOL，所以导致很多旧版本内容已经不再同步，从而导致安装失败，我解决的办法是增加多个镜像冗余，最后问题得以解决，镜像冗余的添加方法是在.repo 文件中添加多个 baseurl。

4.6.集群节点配置、通信与免密互信。

首先，我的三台虚拟主机名分别是 centos7-070(master)、centos7-071(slave)、centos7-072(slave)。而后在 centos7-070 上修改域名映射，修改结果如图 25 所示。

```

127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
# --- BEGIN PVE ---
10.46.1.70 centos7-070.cumtcs.net centos7-070
# --- END PVE ---
10.46.1.71 centos7-071.cumtcs.net centos7-071
10.46.1.72 centos7-072.cumtcs.net centos7-072
~

```

图 25 修改域名映射

而后进行节点间拷贝文件的测试，测试的结果如图 26 所示，我们把 centos7-070 上的 hello.txt 文件传输到 centos7-071 上,可以

看到，这时传输文件是需要 centos7-071 的密码的。但是我在进行第一次测试时并没有成功，因为我只设置了 centos7-070 上的映射，而剩下两个 slave 节点没有设置从而导致传输失败。

```

[root@centos7-070 ~]# scp -P 11071 hello.txt centos7-071@192.168.46.69:/home/centos7-071/
centos7-071@192.168.46.69's password:
hello.txt
[root@centos7-070 ~]#
100% 40 132.5KB/s 00:00

```

图 26 文件传输

而后我们设置节点间两两免密登录，由于设置的步骤有三次重复，因此这里仅仅展示 master 和 slave 间免密登录的设置过程，剩下的设置只需要对剩下的两个节点分别做一次即可。设置的步骤大致如下：1. 在 master 端生成密钥对；2. 在 master 端将公钥 id_rsa.pub 追加到授权的 key 中；3. 在 master 端将公钥发送给 slave；4. 在 slave 端将公钥追加到授权的 key 中。设置的结果和流程如图 27 和图 28 所示。

```

[root@centos7-070 ~]# cd ~/.ssh
[root@centos7-070 .ssh]# ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa.
Your public key has been saved in /root/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:DlcmZhUzh3yCeVdtLPSUYZUL4Ripzn7L8ck9BmIu0wc root@centos7-070
The key's randomart image is:
+---[RSA 2048]---+
  ++=O+=*X|
  o.*.++o*+|
  +.O=. .O.|
  o+.
  .So
  + oE .
  ..+ + .
  o.+*=
  o.+*= o|
+---[SHA256]-----+
[root@centos7-070 .ssh]# touch ~/.ssh/authorized_keys
[root@centos7-070 .ssh]# cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
[root@centos7-070 .ssh]# scp ~/.ssh/id_rsa.pub centos7-071:~/.ssh
The authenticity of host 'centos7-071 (10.46.1.71)' can't be established.
ECDSA key fingerprint is SHA256:2cNWrmdWAUsvy6XYDDQN0DLK5zCP53VyVvKSCDQ/0+bY.
ECDSA key fingerprint is MD5:8f:9f:a5:a5:5e:7f:9f:c7:39:e8:35:a6:8e:d4:35:d7.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'centos7-071,10.46.1.71' (ECDSA) to the list of known hosts.
root@centos7-071's password:
id_rsa.pub
100% 398 2.4MB/s 00:00

```

图 27 免密登录设置 1

```

[root@centos7-071 ~]# touch ~/.ssh/authorized_keys
[root@centos7-071 ~]# cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
[root@centos7-071 ~]# ls ~/.ssh
authorized_keys  bdm_rsa.pub  id_rsa.pub

```

图 28 免密登录设置 2

4.7.配置环境变量与安装 Java 开发环境

首先我们下载 JDK1.8 解压到/opt/modules 下，并在/etc/profile 中添加系统环境变量，添加信息如图 29 所示。

```
export PATH USER LOGNAME MAIL HOSTNAME HISTSIZE HISTCONTROL
export CLASSPATH=../JAVA_HOME/lib;../JAVA_HOME/jre/lib
export JAVA_HOME="/opt/modules/jdk1.8.0_181"
export PATH=../JAVA_HOME/bin:$PATH
```

图 29 添加系统环境变量

而后输入 source /etc/profile 进行启用环境变量，而后我们测试 java 环境是否部署成功，如图 30 所示，发现是成功的。并检查 java 进程是否启动，输入 jps，发现有一个 jps 进程。

```
[root@centos7-070 ~]# source /etc/profile
[root@centos7-070 ~]# java -version
openjdk version "1.8.0_412"
OpenJDK Runtime Environment (build 1.8.0_412-b08)
OpenJDK 64-Bit Server VM (build 25.412-b08, mixed mode)
[root@centos7-070 ~]#
```

图 30 查看 java 版本信息

4.8. ssh 客户端远程操作虚拟机。

ssh [-l login_name] [-p port] [user@]hostname 是 ssh 命令的基本用法。现在我们尝试用 centos7-070 去连接 centos7-071.

```
[root@centos7-070 ~]# ssh root@centos7-071
Last login: Tue Nov 18 09:15:45 2025 from 10.3.29.82
[root@centos7-071 ~]#
```

图 31 ssh 连接 071

如图 31 所示，我们无需进行登录操作，并且可以直接使用 centos7-071 的主机名，因为在前文中我们已经设置过主机名与 ip 地址的映射以及主机间免密通信。

4.9. Docker&Kubernetes 安装部署

5. 结论与讨论

通过本次 Linux 环境实践实验，我从最基础的 Shell 命令入手，逐步掌握了系统操作、文件管理、用户管理、编辑工具使用、软件安装配置、集群环境搭建及远程运维等关键技能。实验初期对路径操作、vim 模式切换等概念理解不够清晰，但在反复练习与错误排查中逐渐掌握。尤其是在集群节点配置与免密互信中，因疏忽未同步所有节点配置导致通信失败，这一经历让我深刻体会到系统配置的全局性与细节的重要性。通过安装 Java 环境、配置多镜像源解决 CentOS 7 软件安装问题等实践，我不仅提升了动手能力，也锻炼了自主解决问题的能力。整体上，本次实验让我真正走入了 Linux 系统世界，理解了其架构与操作逻辑，为后续大数据环境搭建奠定了坚实基础。

实验二 Hadoop 安装部署与开发

1. 实验基本信息

实验名称：Hadoop 安装部署与开发

实验日期：2025.9.28

实验地点：计算机楼

2. 实验目标和实验环境

实验目标：掌握 Hadoop 分布式计算框架的安装和部署

实验环境：MobaXterm、CentOS7

3. 实验内容

1.单机本地部署 Hadoop; 2.伪分布部署 Hadoop; 3.使用 HDFS 命令; 4.分布式部署 Hadoop; 5.MapReduce 编程实例：词频统计; 6.MapReduce 编程实例：排序。

4. 实验步骤和结果

4.1. 单机本地部署 Hadoop

下载的过程很简单不再赘述。而后我们检查 hadoop 是否可用。如图 32 所示。Hadoop 版本为 2.10.0。

```
[root@centos7-070 hadoop]# ./bin/hadoop version
Hadoop 2.10.0
Subversion ssh://git.corp.linkedin.com:29418/hadoop/hadoop.git -r e2f1f118e465e787d8567dfa6e2
Compiled by jhung on 2019-10-22T19:10Z
Compiled with protoc 2.5.0
From source with checksum 7b2d8877c5ce8c9a2cca5c7e81aa4026
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-2.10.0.jar
```

图 32 检查 Hadoop 版本信息

Hadoop 默认本地模式，无需进行配置即可运行,这里我们测试 hadoop 自带的两个样例。首先是计算 PI 值，我们输入 `hadoop jar ./share/hadoop/mapreduce/hadoop-mapreduce-examples-2.10.0.jar pi 2 1000`，结果如图 33。

```
Job Finished in 1.757 seconds
Estimated value of Pi is 3.14400000000000000000
```

图 33 计算 Pi 值结果

而后是统计词频，我们输入 `hadoop jar ./share/hadoop/mapreduce/hadoop-mapreduce-examples-2.10.0.jar wordcount RE-ADME.txt output`，部分结果如图 34 所示。

```
[root@centos7-070 hadoop]# cat output/part-r-00000
(BIS), 1
(ECCN) 1
(TSU) 1
(see 1
5D002.C.1, 1
740.13) 1
<http://www.wassenaar.org/> 1
```

图 34 词频统计结果

4.2. 伪分布部署 Hadoop

配置 core 和 hdfs 的过程不多赘述，不过在配置 core 的时候要注意 hdfs:后要填写自己的 master 节点的主机名或者 ip 地址。

第一次启动时需要执行 NameNode 节点格式化。如图 35。

```
[root@centos7-070 hadoop]# ./bin/hdfs namenode -format
25/10/13 07:56:16 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG: host = centos7-070.cumtcs.net/10.46.1.70
STARTUP_MSG: args = [-format]
STARTUP_MSG: version = 2.10.0
```

图 35 NameNode 节点格式化

启动 Hadoop，输入 `./sbin/start-dfs.sh`，并检查进程情况：

```
[root@centos7-070 hadoop]# ./sbin/start-dfs.sh
Starting namenodes on [localhost]
localhost: starting namenode, logging to /usr/local/hadoop/logs/hadoop-root-namenode-0.0.0.0
localhost: starting datanode, logging to /usr/local/hadoop/logs/hadoop-root-datanode-0.0.0.0
Starting secondary namenodes [0.0.0.0]
0.0.0.0: starting secondarynamenode, logging to /usr/local/hadoop/logs/hadoop-root-secondarynamenode-0.0.0.0
[root@centos7-070 hadoop]# jps
195538 DataNode
195381 NameNode
195899 Jps
195740 SecondaryNameNode
```

图 36 进程情况

执行 Hadoop 自带的应用程序，先上传数据到 HDFS，而后我们运行 Hadoop 自带的 grep 程序，从所有文件中筛选出符合正则表达式 “dfs[a-z.]+" 的单词，并统计出现次数，最后把统计结果输出到 “output” 文件夹中，部分结果如图 37 所示。

```
[root@centos7-070 hadoop]# ./bin/hdfs dfs -ls /output
Found 2 items
-rw-r--r-- 1 root supergroup 0 2025-10-13 08:03 /output/_SUCCESS
-rw-r--r-- 1 root supergroup 77 2025-10-13 08:03 /output/part-r-00000
[root@centos7-070 hadoop]# ./bin/hdfs dfs -cat /output/part-r-00000
1
dfsadmin
1
dfs.replication
1
dfs.namenode.name.dir
1
dfs.datanode.data.dir
```

图 37 grep 运行结果

4.3. 使用 HDFS 命令

首先是目录操作，我们先创建一个新目录，而后显示该目录下的内容，测试的结果如图 38 所示。

```
[root@centos7-070 hadoop]# ./bin/hdfs dfs -mkdir -p /user/hadoop
[root@centos7-070 hadoop]# ./bin/hdfs dfs -ls /user/hadoop
[root@centos7-070 hadoop]# ./bin/hdfs dfs -ls /
Found 3 items
drwxr-xr-x - root supergroup 0 2025-10-13 08:01 /input
drwxr-xr-x - root supergroup 0 2025-10-13 08:03 /output
drwxr-xr-x - root supergroup 0 2025-10-13 08:35 /user
```

图 38 目录操作测试结果 1

我们分别在 /user/hadoop 和 / 目录下面创建一个 input，如果我们删除 /input，则使用 ./bin/hdfs dfs -rm -r /input，加上 -r 的意思是删除目录及其下的所有内容，如图 39 所示。

```
[root@centos7-070 hadoop]# ./bin/hdfs dfs -mkdir /user/hadoop/input
[root@centos7-070 hadoop]# ./bin/hdfs dfs -rm -r /input
Deleted /input
```

图 39 目录操作测试结果 2

对于文件操作，我们创建 myLocalFile.txt，并把它上传到 HDFS，而后我们使用 ls 命令查看一下文件是否成功上传到 HDFS

中，如图 40 所示。

```
[root@centos7-070 hadoop]# ./bin/hdfs dfs -put /home/myLocalFile.txt /user/hadoop/input
[root@centos7-070 hadoop]# ./bin/hdfs dfs -ls /user/hadoop/input
Found 1 items
-rw-r--r-- 1 root supergroup      14 2025-10-13 08:48 /user/hadoop/input/myLocalFile.txt
```

图 40 文件操测试结果 1

使用 cat 命令可以查看 HDFS 中的 myLocalFile.txt 这个文件的内容。而后我们把 HDFS 中的 myLocalFile.txt 文件用 get 下载到本地/目录。使用 ls 查看本地/目录是否有下载下来的 myLocalFile.txt，如图 41 所示。

```
[root@centos7-070 hadoop]# ./bin/hdfs dfs -get /user/hadoop/input/myLocalFile.txt /
[root@centos7-070 hadoop]# ls /
15  bin  dev  file  hello.txt  home  lib  lost+found  mnt
15:26 boot  etc  hadoop-2.10.0.tar.gz  hey.txt  learn.tar  lib64  media  myLocalFile.txt
```

图 41 文件操作测试结果 2

我们还可以将 myLocalFile.txt 拷贝到/input 下，如图 42。

```
[root@centos7-070 hadoop]# ./bin/hdfs dfs -cp /user/hadoop/input/myLocalFile.txt /input
[root@centos7-070 hadoop]# ./bin/hdfs dfs -ls /input
-rw-r--r-- 1 root supergroup      14 2025-10-13 08:51 /input
```

图 42 文件操作测试结果 3

4.4. 分布式部署 Hadoop

在 slaves 文件中把 localhost 删去并添上 centos7-071 和 centos7-072。在第一次实验时，由于我没有删去 localhost，从而导致了在启动 hadoop 后，三个节点同时都是 datanode，这与我们预期是不符的，由此我学习到 slaves 中控制的是 datanode 的节点。

而后修改 core 和 hdfs 文件，core 与前文一致，hdfs 需要注意的就是天上 http-address: localhost:50090，副本数取 2。接着修改 mapred-site.xml 文件，如图 43 所示。


```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.jobhistory.address</name>
    <value>centos7-070:10020</value>
  </property>
  <property>
    <name>mapreduce.jobhistory.webapp.address</name>
    <value>centos7-070:19888</value>
  </property>
</configuration>
```

图 43 mapred-site.xml 配置结果

最后修改 yarn-site.xml 文件，如图 44 所示。

```
<configuration>
  <!-- Site specific YARN configuration properties -->
  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>centos7-070</value>
  </property>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

图 44 yarn-site.xml 配置结果

在 master 节点上将配置好的 Hadoop 软件分发到集群其他节点上，并且在 master 和任何想要直接运行 Hadoop 命令的节点上配置 PATH 环境变量。

我们分别启动 HDFS、yarn、historyserver。输入 jps，查看进程情况，如图 45 所示。

```
[root@centos7-070 hadoop]# jps
316710 ResourceManager
317184 JobHistoryServer
316816 NodeManager
316467 SecondaryNameNode
317309 Jps
316127 NameNode
316280 DataNode
```

图 45 进程情况

通过 Web 界面查看 Hadoop 集群状况，如图 46 所示。

In operation

Show 25 entries Search:

Node	Http Address	Last contact	Last Block Report	Capacity	Blocks	Block pool used	Version
centos7-071.cumtcs.net:50010 (10.46.1.71:50010)	http://centos7-071.cumtcs.net:50075	2s	10m	48.91 GB	0	28 KB (0%)	2.10.0
centos7-072.cumtcs.net:50010 (10.46.1.72:50010)	http://centos7-072.cumtcs.net:50075	2s	10m	48.91 GB	0	28 KB (0%)	2.10.0

Showing 1 to 2 of 2 entries

Previous 1 Next

图 46 Hadoop Web 页面

由于我们实验的平台是远程连接的虚拟机，因此，直接在自己的主机上访问 centos7-070:50090 是不会成功的，第一次遇到这种问题的时候调试很长时间都不明白为什么，最后我通过在我的 windows 主机上用 ssh 远程连接 centos7-070：ssh -L 9000:centos7-070:50090 root@192.168.46.69 -p 11070，而后访问 localhost:9000 成功。

运行 mapreduce 作业这与第 3 小节的操作类似，先将数据上传 HDFS，执行 mapreduce 作业，我们查看最后结果，如图 47。

```
[root@centos7-070 ~]# hdfs dfs -ls /output
Found 2 items
-rw-r--r--  2 root supergroup          0 2025-11-05 00:19 /output/_SUCCESS
-rw-r--r--  2 root supergroup       107 2025-11-05 00:19 /output/part-r-00000
[root@centos7-070 ~]# hdfs dfs -cat /output/part-r-00000
dfsadmin
1
dfs.replication
1
dfs.namenode.secondary.http
1
dfs.namenode.name.dir
1
dfs.datanode.data.dir
```

图 47 mapreduce 运行结果

通过 Web 界面查看 YARN 集群的运行情况我们使用 ssh -L 8088:centos7-070:8088 root@192.168.46.69 -p 11070 远程连接，而后访问 localhost:8088/cluster，可以看到如下集群情况以及 mapreduce 作业完成情况。

Node Labels	Rack	Node State	Node Address	Node HTTP Address	Last health-update
	/default-rack	RUNNING	centos7-071.cumtcs.net:42683	centos7-071.cumtcs.net:8042	Mon Dec 08 09:45:26 +0000 2025
	/default-rack	RUNNING	centos7-072.cumtcs.net:35361	centos7-072.cumtcs.net:8042	Mon Dec 08 09:45:26 +0000 2025

图 48 集群节点情况

ID	User	Name	Application Type	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus
application_1763446166836_0001	root	importsv_student	MAPREDUCE	default	0	Tue Nov 18 19:30:12 +0800 2025	Tue Nov 18 19:30:13 +0800 2025	Tue Nov 18 19:30:25 +0800 2025	FINISHED	SUCCEEDED

图 49 作业完成情况

4.5. MapReduce 编程实例：词频统计

编写程序的过程略，编写完用：javac WordCount.java 来编译，编译完成会生成多个 class 文件，而后将.class 文件打包成 jar 包：运行 jar -cvf WordCount.jar ./WordCount*.class，结果如图 50。

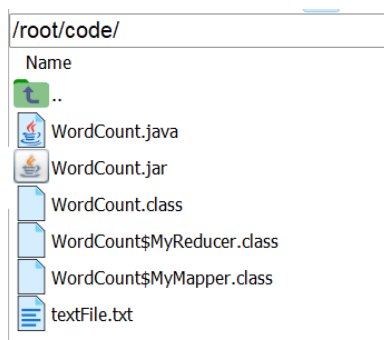


图 50 编译结果

在 Hadoop 运行环境中运行 jar 包，并使用 cat 命令查看部分结果，如图 51。

```
[root@centos7-070 code]# hdfs dfs -cat /outputFile/part-r-00000
apple 1
banana 1
guitar 1
jazz 1
music 1
```

图 51 单词计数结果

4.6. MapReduce 编程实例：排序

先使用 maven 创建项目，配置依赖，在 IDEA 中，修改 pom.xml 文件，添加 hadoop-common、hadoop-hdfs、hadoop-client，版本均为 2.10.0。

编写完代码，我们进入项目根目录执行：mvn clean package，可以得到 goodsort-1.0-SNAPSHOT.jar 包

提交集群运行程序，先将预先准备好的数据上传至 HDFS；如果有/output 文件夹要先删掉；而后将程序提交到集群，我们用 cat 命令查询运行结果，部分结果如图 52 所示。

```
[root@centos7-070 code]# hdfs dfs -cat /output/part-r-00000
96      1010603
96      1010178
97      1010637
97      1010152
100     1010037
100     1010102
100     1010037
103     1010320
104     1010510
104     1010280
```

图 52 goodsort 结果

5. 结论与讨论

基于实验二的 Hadoop 安装部署与开发实践，我从单机本地部署开始逐步深入至伪分布式和完全分布式集群的搭建，在配置过程中遇到并解决了多个典型问题，如 slaves 文件配置不当导致节点角色异常、Web UI 需要远程连接才能正常访问等，通过反复调试和查阅资料掌握了关键配置文件的修改逻辑与集群启动顺序。MapReduce 编程实例中，我亲手实现了词频统计与排序算法，从环境搭建、代码编写、编译打包到集群提交与结果验证，完整经历了大数据应用

开发的典型流程。这次实验让我深刻体会到分布式系统的复杂性不仅在于技术本身，更在于环境的一致性和配置的精确性，任何一个细节的疏忽都可能导致整个集群运行异常。通过动手实践，我不仅学会了 Hadoop 的核心组件与操作命令，更培养了系统思维和故障排查能力，为后续学习更复杂的大数据框架打下了坚实基础。

实验三 Spark 安装部署与开发

1. 实验基本信息

实验名称：Spark 安装部署与开发

实验日期：2025.10.12

实验地点：计算机楼

2. 实验目标和实验环境

实验目标：熟练掌握 Spark 安装部署与开发的流程

实验环境：是 MobaXterm、CentOS7

3. 实验内容

1. Spark 的 Local 部署; 2. Spark 的 Standalone 部署; 3. Spark 的 Yarn 部署; 4. Spark shell 与 RDD 基本操作; 5. Spark shell 编程实例; 6. SBT 工具; 7. Spark 编程实例：排序

4. 实验步骤和结果

4.1 Spark 的 Local 部署

下载安装 Scala, Spark 并配置环境变量，配置与 Java 类似。运行 Spark Shell，在 Spark shell 中做一些测试，如图 53 所示。

```
scala> val rdd=sc.textFile("file:/root/software/spark/README.md")
rdd: org.apache.spark.rdd.RDD[String] = file:/root/software/spark/README.md MapPartitionsRDD[1] at textFile at <console>:24

scala> rdd.count()
res0: Long = 108

scala> val wordCounts = rdd.flatMap(line => line.split(" ")).map(word => (word, 1)).reduceByKey((a, b) => a + b)
wordCounts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[4] at reduceByKey at <console>:25

scala> wordCounts.collect()
res1: Array[(String, Int)] = Array((package,1), (this,1), (integration,1), (Python,2), (cluster,1), (its,1), ([run,1), (There,1), (general,2), (have,1), (pre-built,1), (Because,1), (YARN,1), (locally,2), (changed,1), (locally,1), (several,1), (only,1), (Configuration,1), (This,2), (basic,1), (first,1), (learning,1), (documentation,3), (graph,1), (Hive,2), (info,1), (["Specifying,1), ("yarn",1), ([params]'.1), ([project,1), (prefer,1), (SparkPi,2), (engine,2), (version,1), (file,1), (documentation,1), (MASTER,1), (example,3), (are,1), (systems,1), (params,1), (scala>,1), (DataFrames,1), (provides,1), (refer,2), (configure,1), (Interactive,2), (R,1), (can,6), (build,3), (when,1), (easiest,1), (Maven](https://maven.apache.org/),1), (Apache,1), (guide](ht...
```

图 53 Spark Shell 的测试

部署 Spark 本地伪分布式集群，Hadoop 伪分布式集群以部署，首先配置 slaves 文件，由于这里是本地伪分布式集群，因此默认是 localhost。其次配置 spark-env.sh 文件，尤其注意 SPARK_MASTER_IP=127.0.0.1

而后启动 Hadoop。查看进程信息，如图 54，可以看到我们成功启动，这里 master 节点兼任 datanode。

```
[root@centos7-070 hadoop]# jps
570040 WrapperSimpleApp
819038 DataNode
819705 Jps
819625 Worker
819241 SecondaryNameNode
818889 NameNode
608974 JobHistoryServer
819536 Master
```

图 54 进程情况

在本地伪集群中做一些实验。先上传数据文件到 HDFS，而后启动 spark shell 窗口，先定义一个 mytxt，调用 mytxt.count()。从图 55 中看出，最后 count 结果为 108。

```
Using Scala version 2.12.10 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_181)
Type in expressions to have them evaluated.
Type :help for more information.

scala> var mytxt = sc.textFile("hdfs://localhost:9000/myspark/README.md")
mytxt: org.apache.spark.rdd.RDD[String] = hdfs://localhost:9000/myspark/README.md MapPartitionsRDD[1] at textFile at <console>:24

scala> mytxt.count()
res0: Long = 108
```

图 55 count 运行结果

我们通过 Spark 的 Web 界面观察集群运行情况，同样用 ssh 在 Windows 远程连接 centos7-070：ssh -L 9001:centos7-070:8080 root@192.168.46.69 -p 11070，我们访问 localhost:9001。

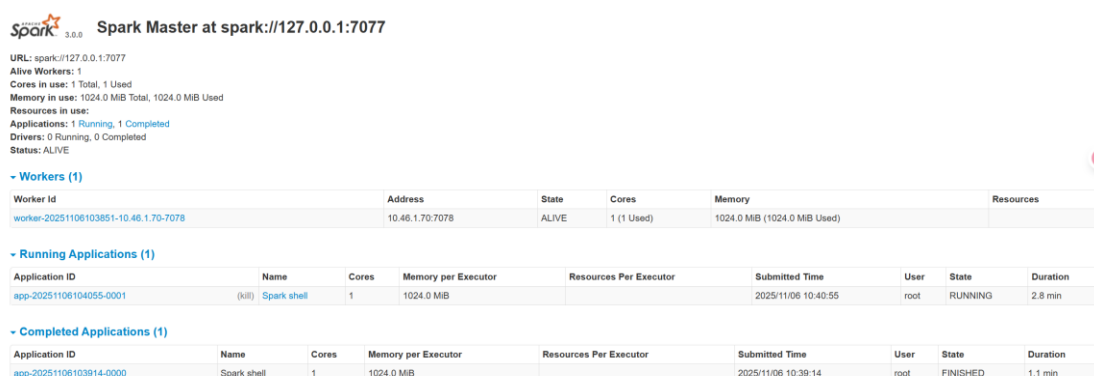


图 56 Spark Web 页面

4.2 Spark 的 Standalone 部署

编辑 slaves 文件，添加所有 Worker 节点，这里的 slaves 包括了 centos7-070 Master 节点，在第一次实验时，没有添加 master 节点，这不符合 Standalone 的部署模式。

编辑 spark-env.sh，配置环境变量，这里与之前的伪分布式不同的就是 SPARK_MASTER_HOST 改为 centos7-070。编辑 spark-

defaults.conf，配置默认模式：spark.master spark://node01:7077。

从 master 节点将 spark 目录分发到其他节点。

运行 Spark-shell，而后在集群中做一些实验这里的实验过程与 4.1 的过程一致，不再过多赘述。实验的结果如图 57 所示。

```
scala> var mytxt = sc.textFile("hdfs://centos7-070:9000/myspark/README.md")
mytxt: org.apache.spark.rdd.RDD[String] = hdfs://centos7-070:9000/myspark/README.md MapPartitionsRDD[3] at textFile at <console>:24

scala> var mytxt = sc.textFile("hdfs://centos7-070:9000/myspark/README.md")
mytxt: org.apache.spark.rdd.RDD[String] = hdfs://centos7-070:9000/myspark/README.md MapPartitionsRDD[5] at textFile at <console>:24

scala> mytxt.count()
res1: Long = 108
```

图 57 shell 中运行结果

通过 Spark 的 Web 界面观察集群运行情况同样用 ssh 远程在 Windows 主机连接 centos7-070：ssh -L 9001:centos7-070:8080 root@192.168.46.69 -p 11070，我们访问 localhost:9001。可以看到 Worker 有三个节点包括 Master。

 **Spark Master at spark://centos7-070:7077**

URL: spark://centos7-070:7077
 Alive Workers: 3
 Cores in use: 3 Total, 3 Used
 Memory in use: 3.0 GiB Total, 3.0 GiB Used
 Resources in use:
 Applications: 1 Running, 0 Completed
 Drivers: 0 Running, 0 Completed
 Status: ALIVE

Workers (3)

Worker Id	Address	State	Cores	Memory	Resources
worker-20251106114106-10.46.1.71-7078	10.46.1.71:7078	ALIVE	1 (1 Used)	1024.0 MB (1024.0 MB Used)	
worker-20251106114109-10.46.1.70-7078	10.46.1.70:7078	ALIVE	1 (1 Used)	1024.0 MB (1024.0 MB Used)	
worker-20251106114109-10.46.1.72-7078	10.46.1.72:7078	ALIVE	1 (1 Used)	1024.0 MB (1024.0 MB Used)	

Running Applications (1)

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20251106122427-0000	(kill) Spark shell	3	1024.0 MB		2025/11/06 12:24:27	root	RUNNING	2.9 min

图 58 Spark Web 页面

4.3 Spark 的 Yarn 部署

修改 Yarn 集群配置，添加 pmem_check_enabled 和 vmem_check_enabled 为 false，修改完将 centos7-070 上 yarn-site.xml 文件分发到其他节点

启动 HDFS & YARN 集群。运行 Spark shell 做一些实验，过程

与 4.2 完全一致，结果如图 59 所示。

```
scala> var mytxt = sc.textFile("hdfs://centos7-070:9000/myspark/README.md")
mytxt: org.apache.spark.rdd.RDD[String] = hdfs://centos7-070:9000/myspark/README.md Ma
pPartitionsRDD[1] at textFile at <console>:24

scala> mytxt.count()
res0: Long = 108
```

图 59 count 运行结果

通过 Client 模式提交运行应用程序，计算 Pi 值，结果如图 60

```
Pi is roughly 3.1413875141387515
25/11/06 12:49:24 INFO server.AbstractConnector: Stopped Spark@2cd0f5a3{HTTP/1.1,[http/1.1]}{0.0.0.0:4040}
25/11/06 12:49:24 INFO ui.SparkUI: Stopped Spark web UI at http://centos7-070.cumtcs.net:4040
25/11/06 12:49:24 INFO cluster.YarnClientSchedulerBackend: Interrupting monitor thread
25/11/06 12:49:25 INFO cluster.YarnClientSchedulerBackend: Shutting down all executors
25/11/06 12:49:25 INFO cluster.YarnSchedulerBackend$YarnDriverEndpoint: Asking each executor to shut down
25/11/06 12:49:25 INFO cluster.YarnClientSchedulerBackend: YARN client scheduler backend Stopped
25/11/06 12:49:25 INFO spark.MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
25/11/06 12:49:25 INFO memory.MemoryStore: MemoryStore cleared
25/11/06 12:49:25 INFO storage.BlockManager: BlockManager stopped
25/11/06 12:49:25 INFO storage.BlockManagerMaster: BlockManagerMaster stopped
25/11/06 12:49:25 INFO scheduler.OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
25/11/06 12:49:25 INFO spark.SparkContext: Successfully stopped SparkContext
25/11/06 12:49:25 INFO util.ShutdownHookManager: Shutdown hook called
25/11/06 12:49:25 INFO util.ShutdownHookManager: Deleting directory /tmp/spark-1d43b6bd-fbcc-4651-8f7d-7f9fe9050cd6
25/11/06 12:49:25 INFO util.ShutdownHookManager: Deleting directory /tmp/spark-6e089fdf-6ac8-4449-90e8-2d94e1d04e02
```

图 60 Client 模式运行结果

通过 Cluster 模式提交运行一个应用程序，计算 Pi 值，这里的区别就是将(5)中的 deploy-mode 改为 cluster，结果如图 61。

```
client token: N/A
diagnostics: N/A
ApplicationMaster host: centos7-072.cumtcs.net
ApplicationMaster RPC port: 34993
queue: default
start time: 1762433521265
final status: SUCCEEDED
tracking URL: http://centos7-070:8088/proxy/application_1762432711962_0003/
user: root
25/11/06 12:52:16 INFO util.ShutdownHookManager: Shutdown hook called
25/11/06 12:52:16 INFO util.ShutdownHookManager: Deleting directory /tmp/spark-bdd504e2-1f87-4b7f-b39a-2cb5ef00a246
25/11/06 12:52:16 INFO util.ShutdownHookManager: Deleting directory /tmp/spark-b0e3cb8d-b42b-4364-8d27-553590b7a3e8
```

图 61 cluster 模式的运行结果

在操作过程中，随时可以通过 Web 界面查看 YARN 集群的运行情况，我们可以看到在 Web 界面中有尾号为 0003 的作业成功。

application_1762432711962_0003	root	org.apache.spark.examples.SparkPi	SPARK	default	0	Thu Nov 6 20:52:01 +0800 2025	Thu Nov 6 20:52:01 +0800 2025	Thu Nov 6 20:52:15 +0800 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A
application_1762432711962_0002	root	Spark Pi	SPARK	default	0	Thu Nov 6 20:49:12 +0800 2025	Thu Nov 6 20:49:12 +0800 2025	Thu Nov 6 20:49:25 +0800 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A
application_1762432711962_0001	root	Spark shell	SPARK	default	0	Thu Nov 6 20:41:57 +0800 2025	Thu Nov 6 20:41:58 +0800 2025	Thu Nov 6 20:44:15 +0800 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A

图 62 Web 界面的任务完成情况

4.4 Spark shell 与 RDD 基本操作

创建 RDD，以 Spark 安装目录中的“README.md”文件作为数据源新建一个 RDD，如图 63 所示。

```
scala> val textFile=sc.textFile("file:///root/software/spark/README.md")
textFile: org.apache.spark.rdd.RDD[String] = file:///root/software/spark/README.md MapPartitionsRDD[3] at textFile at <console>:24
```

图 63 新建 RDD

使用 action API - count()可以统计该文本文件的行数。

```
scala> textFile.count()
res1: Long = 108
```

图 64 调用 count API

使用 transformation API - filter()可以筛选出只包含 Spark 的行。从图 65 中可以看出有 19 行。

```
scala> val lineWithSpark=textFile.filter(line⇒line.contains("Spark"))
lineWithSpark: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[4] at filter at <console>:25
scala> lineWithSpark.count()
res2: Long = 19
```

图 65 调用 filter API

使用链式操作，在一条代码中同时使用多个 API 连续运算。

```
scala> val lineWithSpark=textFile.filter(line⇒line.contains("Spark")).count()
lineWithSpark: Long = 19
```

图 66 链式操作

实现词频统计，先使用 flatMap()将每一行的文本内容通过空格进行划分为单词，再使用 map()将单词映射为(K,V)的键值对，最后使用 reduceByKey()将相同单词的计数进行相加，最终得到该单词总的出现的次数。结果如图 67 所示。

```
scala> val wordCounts=textFile.flatMap(line⇒line.split(" ").map(word⇒(word,1))).reduceByKey((a,b)⇒a+b)
wordCounts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[8] at reduceByKey at <console>:25

scala> wordCounts.collect()
res3: Array[(String, Int)] = Array((package,1), (this,1), (integration,1), (Python,2), (cluster,1), ([run,1), (There,1), (Because,1), (its,1), (YARN,1), (have,1), (general,2), (pre-built,1), (locally,2), (locally.,1), (changed,1), (several,1), (only,1), (Configuration,1), (This,2), (first,1), (basic,1), (documentation,3), (learning,1), (graph,1), (Hive,2), (info,1), ([Specifying,1), ("yarn",1), ([params]`,1), ([project,1), (prefer,1), (SparkPi,2), (engine,2), (version,1), (file,1), (documentation,,1), (MASTER,1), (example,3), (are,1), (systems.,1), (params,1), (scala>,1), (DataFrames,,1), (provides,1), (refer,2), (configure,1), (Interactive,2), (R,,1), (can,6), (build,3), (when,1), (how,3), (easiest,1), (Maven][https://maven.apache.org/),1), (Apache,1), (...)
```

图 67 词频统计结果

4.5 Spark shell 编程实例

统计用户收藏的商品数量，我们先设计一个用户收藏表。而后使用 client 模式打卡 Spark Shell；而后我们统计用户收藏数据中，每个用户收藏商品的数量，用户收藏数据中都有哪些商品被收藏（去重）。具体的命令与结果如图 68 所示。

```
scala> val rdd = sc.textFile("hdfs:///sparktest/buyer_favorite")
rdd: org.apache.spark.rdd.RDD[String] = hdfs:///sparkTest/buyer_favorite MapPartitionsRDD[1] at textFile at <console>:24

scala> rdd.map(line⇒ (line.split('\t')(0),1)).reduceByKey(_+_).collect
res0: Array[(String, Int)] = Array((20042,1), (20001,2), (10181,1), (20067,1), (20056,2))

scala> rdd.map(line ⇒ line.split('\t')(1)).distinct.collect
res1: Array[String] = Array(1002061, 1000481, 1003290, 1001368, 1001560, 1001597, 1003289)
```

图 68 统计用户收藏商品数量

查询用户购买的商品，首先设计 orders 表和 order_items 表。而后将这两张表数据上传到 HDFS，并用 client 模式打开 Spark Shell；我们先创建两个 RDD，分别加载 orders 文件以及 order_items 文件中的数据；而后对 rdd1 和 rdd2 进行 map 映射，得出计算需要的两列数据；最后将 rdd11 以及 rdd22 中的数据，根据 Key 值进行 Join，输出结果，结果与过程如图 69 所示。

```
scala> val rdd1 = sc.textFile("hdfs:///sparktest/orders");
rdd1: org.apache.spark.rdd.RDD[String] = hdfs:///sparktest/orders MapPartitionsRDD[1] at textFile at <console>:25

scala> val rdd2 = sc.textFile("hdfs:///sparktest/order_items");
rdd2: org.apache.spark.rdd.RDD[String] = hdfs:///sparktest/order_items MapPartitionsRDD[3] at textFile at <console>:25

scala> val rdd11 = rdd1.map(line => (line.split('\t')(0), line.split('\t')(2)))
rdd11: org.apache.spark.rdd.RDD[(String, String)] = MapPartitionsRDD[4] at map at <console>:25

scala> val rdd22 = rdd2.map(line => (line.split('\t')(1), line.split('\t')(2)))
rdd22: org.apache.spark.rdd.RDD[(String, String)] = MapPartitionsRDD[5] at map at <console>:25

scala> val rddresult = rdd11 join rdd22
rddresult: org.apache.spark.rdd.RDD[(String, (String, String))] = MapPartitionsRDD[8] at join at <console>:25

scala> rddresult.collect
res0: Array[(String, (String, String))] = Array((52294,(174912,1022245)), (52294,(174912,1014724)), (52294,(174912,1023399)), (52293,(175823,1016840)), (52293,(175823,1014040)))
```

图 69 查询用户购买商品过程与结果

使用 Spark SQL 查询用户购买的商品，我们先使用 Spark SQL API：用 case class 方式定义 RDD，创建 orders 表，先创建 SQL-Context；而后定义 case class；接着读取数据，转换为 DataFrame；最后注册临时表。

在首次实验时，在打印 tableName 时出现了错误，没有找到任何的表，如图 70 中所示，这表明 case class Orders 没有被定义，于是重新定义后，结果正确。

```
scala> sqlContext.sql("show tables").map(t=>"tableName is:"+t(0)).collect().foreach(println)
25/11/07 08:59:45 WARN conf.HiveConf: HiveConf of name hive.stats.jdbc.timeout does not exist
25/11/07 08:59:45 WARN conf.HiveConf: HiveConf of name hive.stats.retries.wait does not exist
25/11/07 08:59:49 WARN metastore.ObjectStore: Version information not found in metastore. hive.metastore.schema.verification is not enabled so recording the schema version 2.3.0
25/11/07 08:59:49 WARN metastore.ObjectStore: setMetaStoreSchemaVersion called but recording version is disabled: version = 2.3.0, comment = Set by MetaStore root@10.46.1.70
25/11/07 08:59:49 WARN metastore.ObjectStore: Failed to get database default, returning NoSuchObjectException
25/11/07 08:59:49 WARN metastore.ObjectStore: Failed to get database global_temp, returning NoSuchObjectException
tableName is:

scala> sqlContext.sql("select order_id,buyer_id from orders").collect
res3: Array[org.apache.spark.sql.Row] = Array([52293,175823], [52294,174912], [52295,176105])
```

图 70 查询表时出错

再而我们用 applyScheme 方式定义 RDD，创建 order_items 表。首先我们导入两项依赖；而后构建 order_Items 表；接着构建 schema；最后验证临时表创建成功，结果如图 71。

```
scala> spark.sql("SHOW TABLES").show()
+-----+-----+-----+
|database|tableName|isTemporary|
+-----+-----+-----+
|         |order_items|          true|
|         |orders|          true|
+-----+-----+-----+
```

图 71 查询表时正确

我们对 order 表及 order_items 表做 join 操作，统计有哪些用户购买了什么商品，操作与结果如图 72 所示。

```
scala> sqlContext.sql("select orders.buyer_id, order_items.goods_id from order_items join orders
on order_items.order_id=orders.order_id").collect
res4: Array[org.apache.spark.sql.Row] = Array([176105,1023399], [175823,1016840], [175823,1014040], [174912,1022245], [174912,1014724], [174912,1010731], [174912,1014200], [174912,1001012])
```

图 72 join 操作过程与结果

在 Spark SQL 窗口中直接执行 SQL 脚本，首先用 create table 操作创建两张表：orders 和 order_items，而后用 show tables 查看创建的表，如图 73。

```
spark-sql> show tables;
25/11/07 10:20:26 INFO metastore.HiveMetaStore: 0: get_database: default
25/11/07 10:20:26 INFO HiveMetaStore.audit: ugi=root ip=unknown-ip-addr cmd=get_database: default
25/11/07 10:20:26 INFO metastore.HiveMetaStore: 0: get_database: default
25/11/07 10:20:26 INFO HiveMetaStore.audit: ugi=root ip=unknown-ip-addr cmd=get_database: default
25/11/07 10:20:26 INFO metastore.HiveMetaStore: 0: get_tables: db=default pat=*
25/11/07 10:20:26 INFO HiveMetaStore.audit: ugi=root ip=unknown-ip-addr cmd=get_tables: db=default pat=*
25/11/07 10:20:26 INFO codegen.CodeGenerator: Code generated in 276.828439 ms
default order_items false
default orders false
Time taken: 0.454 seconds, Fetched 2 row(s)
25/11/07 10:20:26 INFO thriftserver.SparkSQLCLIDriver: Time taken: 0.454 seconds, Fetched 2 row(s)
```

图 73 查询有哪些表

接着我们将 HDFS 的 orders 表和 order_items 表中数据加载到创建的两个表中。我们验证数据是否加载成功，如图 74 图 75 所示。

```
52293 111215052627 175823 2011-12-15 02:35:17
52294 111215052626 174912 2011-12-15 02:18:45
52295 111215052625 176105 2011-12-15 01:52:33
Time taken: 4.659 seconds, Fetched 3 row(s)
25/11/07 10:19:51 INFO thriftserver.SparkSQLCLIDriver: Time taken: 4.659 seconds, Fetched 3 row(s)
```

图 74 orders 表内容

```
252578 52293 1016840
252579 52293 1014040
252580 52294 1014200
252581 52294 1001012
252582 52294 1022245
252583 52294 1014724
252584 52294 1010731
252586 52295 1023399
Time taken: 0.259 seconds, Fetched 8 row(s)
25/11/07 10:24:20 INFO thriftserver.SparkSQLCLIDriver: Time taken: 0.259 seconds, Fetched 8 row(s)
```

图 75 order_items 表内容

对 order 表及 order_items 表做 join 操作，直接使用 sql 语句进行表的自然连接，查询结果如图 76。

```
175823 1016840
175823 1014040
174912 1014200
174912 1001012
174912 1022245
174912 1014724
174912 1010731
176105 1023399
Time taken: 3.903 seconds, Fetched 8 row(s)
25/11/07 10:37:01 INFO thriftserver.SparkSQLCLIDriver: Time taken: 3.903 seconds, Fetched 8 row(s)
```

图 76 表连接后内容

在这里进行 load data 操作时，我出现了一个错误，Spark 不是在 HDFS 上查找文件，而是在本地文件系统上找文件，Spark Executor 在每个 Worker 节点的本地路径去找，而除了 Master 节点，其他节点本地根本没有这个文件，因此本地存储的情况下就会出错。

我是先通过同步所有节点的数据后，再进行的测试，所有的 Worker 节点上都存在 orders 的文件，这样 Executor 任务在其它节点执行时就可成功找到文件。

而最稳妥的方法还是让表存在 HDFS，而不是本地，只需要再 sql 中重新设置一下 spark.sql.warehouse.dir 的路径即可，而后删除表后重建，再次导入数据后问题得以解决。

4.6 使用 SBT 构建工具进行 Spark 应用程序开发

由于验证 sbt 安装没有成功，于是我们更换 sbt 仓库，仓库的修改信息如图 77 所示。而后修改 sbt 文件中的配置文件 sbtopts，在末尾添加：-Dsbt.override.build.repos=true。最后再次尝试使用 sbtVersion 验证 sbt 是否安装成功，测试最终成功。

```
[repositories]
local
huaweicloud-maven: https://repo.huaweicloud.com/repository/maven/
maven-central: https://repo1.maven.org/maven2/
sbt-plugin-repo: https://repo.scala-sbt.org/scalasbt/sbt-plugin-releases, [organization]/[module]/(scala_[scalaVersion]/)(sbt_[sbtVersion]/)[revision]/[type]s/[artifact](-[classifier]).[ext]
```

图 77 更改仓库信息

使用 Scala 语言编写应用程序,创建程序根目录,并创建程序所需文件夹结构/sparkapp/src/main/scala, 而后创建 SimpleApp.scala。接着创建一个 simple.sbt 文件，用于声明该应用程序的信息以及与 Spark 的依赖关系。

使用 sbt 对应用程序进行打包：/usr/local/sbt/sbt package，打包结果如图 78 所示。打包成功。

```
[info] Non-compiled module 'compiler-bridge_2.10' for Scala 2.10.5. Compiling...
[info]   Compilation completed in 12.55s.
[success] Total time: 18 s, completed Nov 7, 2025 11:38:30 AM
```

图 78 使用 sbt 打包结果

最后用 spark-submit 提交应用程序到 Spark 运行，包含 a 的有 64 行，包含 b 的有 32 行。

4.7 使用 Maven 构建工具进行 Spark 应用程序开发

构建项目，配置 Spark 依赖，修改 pom.xml 文件，主要导入 spark-core, spark-hive, spark-sql, hadoop-client，由于参考资料给

出的依赖版本出现了各种情况的依赖冲突、源无法导入的问题，因此给 scala, spark, hadoop 的版本分别选定为 2.11.12, 2.4.8, 2.10.0。测试后该组合效果不错。

编写逻辑代码后编译项目，编写代码部分略，我们在项目根目录对项目进行打包：mvn clean package，可以得到一个 JavaSpark-GoodsCount-1.0-SNAPCHAT.jar 包。

提交集群运行程序，按照基本流程，上传数据、提交程序到 Spark 集群，最后我们通过 cat 命令查看结果，如图 79。

```
[root@centos7-070 bin]# hdfs dfs -cat /sparktest/out/part-00000
(20042,1)
(20001,2)
(10181,1)
(20067,1)
(20056,2)
```

图 79 查询作业结果

这里的依赖版本问题非常严重，在原有的代码中，call 函数需要修改，因为在我的 Spark 2.4.8 中，call 实现的是 FlatMapFunction，而在 Spark2.4.8 中，call 需要返回的是 Iterator<R>而不是 Iterable，修改的具体内容如图 80 所示。

```
@Override
public Iterator<String> call(String s) {
    String[] word = s.split(regex: "\\t", limit: 2);
    return Arrays.asList(word[0]).iterator();
}
```

图 80 修改 call 函数的结果

5. 结论与讨论

基于实验三的 Spark 安装部署与开发实践，我从 Local 模式起

步，逐步实现了 Standalone 和 Yarn 两种集群部署模式，并深入学习了 RDD 操作、Spark SQL 编程以及使用 SBT 与 Maven 构建应用程序的全流程。在配置过程中，我遇到了 slaves 文件遗漏 Master 节点导致 Standalone 模式异常、Spark SQL 表加载因数据存储路径不一致而失败、SBT 仓库源配置错误导致依赖下载失败、Maven 项目因 Spark 版本不匹配引发 API 调用错误等一系列问题。通过逐一排查与调试，我不仅掌握了 Spark 集群的部署逻辑与配置文件的关键作用，还学会了如何根据日志信息定位版本兼容性问题，并灵活调整依赖版本与代码实现。本次实验让我深刻认识到大数据框架的生态复杂性与版本管理的重要性，Spark 强大的内存计算能力与丰富的 API 为数据处理带来了高效与便捷，但同时也要求开发者具备细致的环境配置能力与持续学习的态度。通过动手搭建与开发，我不仅巩固了分布式计算的理论知识，更提升了在实际项目中解决复杂技术问题的综合能力。

实验四 Flink 安装部署与开发

1. 实验基本信息

实验名称：Flink 安装部署与开发

实验日期：2025.10.26

实验地点：计算机楼

2. 实验目标和实验环境

实验目标：掌握 Flink 的安装部署与开发

实验环境：MobaXterm、CentOS7

3. 实验内容

1. Flink 安装部署的环境准备工作; 2. Local 模式安装部署 Flink ;
3. Standalone 模式安装部署 Flink; 4. YARN 模式安装部署 Flink ; 5.
Flink 应用程序开发

4. 实验步骤和结果

4.1 Local 模式安装部署 Flink

本地模式（本地 Shell 窗口），启动 Shell 窗口，并在 Shell 交互式窗口中直接提交一个任务，如图 81，该任务将 words.txt 中的单词转换成键值的形式并打印出来。

```
scala> benv.readTextFile("/root/words.txt").flatMap(_.split(" ")).map((_,1)).groupBy(0).  
sum(1).print()  
(autumn,1)  
(fly,1)  
(guitar,1)  
(jazz,1)  
(leaves,1)  
(me,1)  
(moon,1)  
(the,1)  
(to,1)
```

图 81 任务结果图

本地模式（伪分布式集群）可以在单节点上启动一个 Flink 伪分布式集群，使用 jps 命令可以看到 Flink 的两个进程，TaskManager 和 JobManager 同时在 centos7-070 上运行。如图 82。

```
[root@centos7-070 bin]# jps
570040 WrapperSimpleApp
1056598 Jps
869940 JobHistoryServer
1056105 StandaloneSessionClusterEntrypoint
1056539 TaskManagerRunner
```

图 82 进程情况

可以访问 Flink 的 web 界面（访问方式同前文）。如图 83。

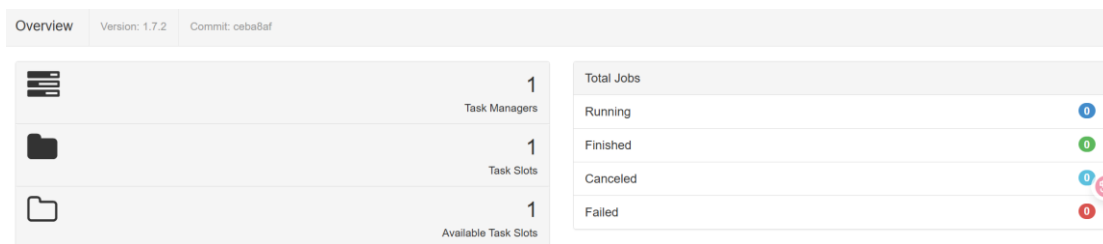


图 83 Flink 的 Web 界面

提交一个测试任务，统计 words.txt 文件中各单词的数量，而后我们 cat 输出文件查看结果，如图 84。

```
[root@centos7-070 ~]# cat /root/out.txt
autumn 1
fly 1
guitar 1
jazz 1
leaves 1
me 1
moon 1
the 1
to 1
```

图 84 words.txt 单词数量

同样的方法，我们查看 Web 界面，可以从图 85 中看到 WordCount 的任务记录。

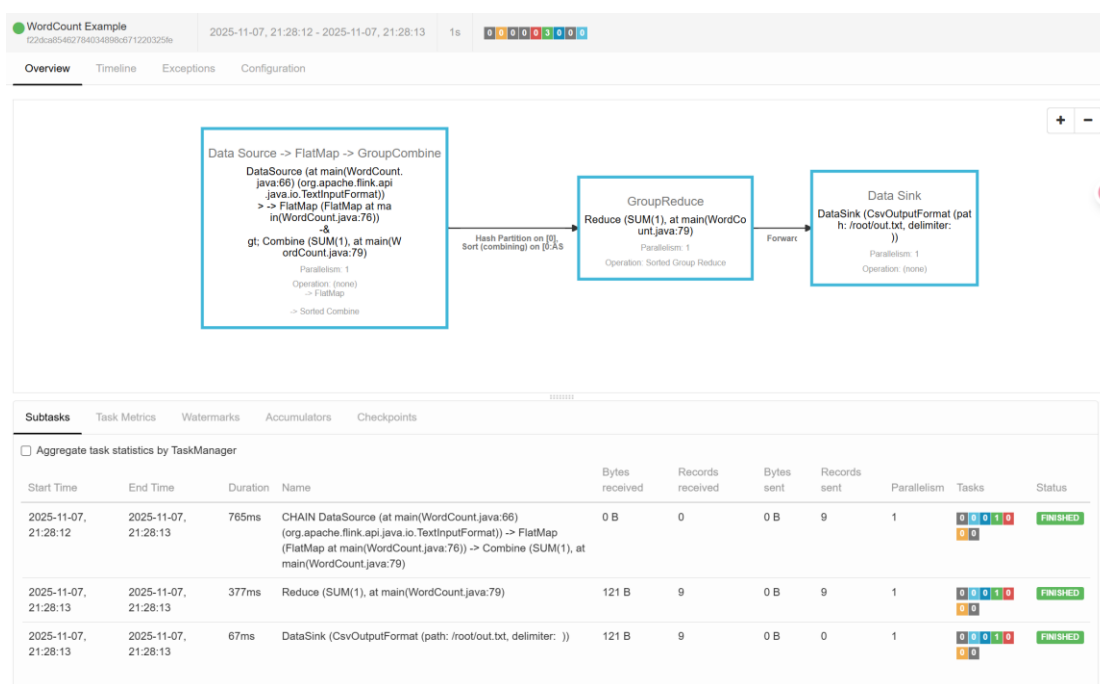


图 85 Web 界面中任务进行情况

4.2 Standalone 模式安装部署 Flink

在 centos7-070 上修改 flink-conf.yaml 文件配置；在 centos7-070 上修改 masters 文件，添加上 centos7-070:8081；在 centos7-070 上修改 slaves 文件，注意三个节点都要添加；在 centos7-070 上修改/etc/profile 设置 Hadoop 配置目录；而后分发 flink 目录及 profile 文件到另外两个节点，这里要发 profile 文件以及 flink 文件，因为要统一所有节点的环境变量信息。并且在其他两个节点上加载环境变量。

启动 Flink 集群，访问 Flink 的 web 界面，观察集群状态和运行情况。结果如图，可以看到有 3 个 TaskManager，对应我们三个节点，剩下各 6 个的 Task Slots 和 Available Task Slots，表示一共有多少个任务槽位，可并行任务的数量。

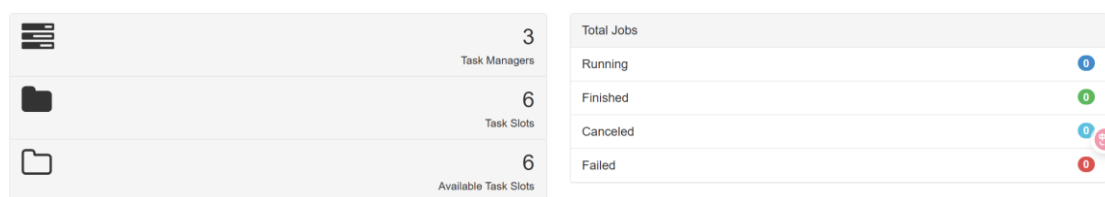


图 86 Web 界面中节点情况

启动 HDFS 集群，上传数据，提交一个测试任务，我们 cat 结果文件可以得到最终输出，如图 87 所示。

```
[root@centos7-070 flink]# ./bin/flink run examples/batch/WordCount.jar --input hdfs://
fs://10.46.1.70:9000/test/output2/result.txt
Starting execution of program
Program execution finished
Job with JobID 0b270e963e2acfc60fa91eb3aec05906 has finished.
Job Runtime: 1250 ms
[root@centos7-070 flink]# hdfs dfs -cat /test/output2/result.txt
1 1
13 1
5d002 1
740 1
about 1
account 1
administration 1
```

图 87 运行结果

再次查询 Web 界面，可以看到 WordCount 任务成功执行。

Start Time	End Time	Duration	Job Name	Job ID	Tasks	Status
2025-11-07, 21:47:27	2025-11-07, 21:47:28	1s	WordCount Example	0b270e963e2acfc60fa91eb3aec05906	3 0 0 0 0 3 0 0 0 0	FINISHED

图 88 Web 界面中 WordCount 执行情况

4.3 Standalone 模式安装部署 Flink

在 Flink Standalone 集群上安装部署 Zookeeper 集群，Zookeeper 需要节点之间两两互信免密，按照完成后，接着在 centos7-070 上修改配置文件 zoo.cfg。

从 centos7-070 上将 zookeeper 分发到其他节点后，在各个节点的 zookeeper/tmp 目录下，分别创建一个名为 myid，内容分别为 1/2/3 的文件（对应于 zoo.cfg 的 server.1/server.2/server.3）。

在各节点上启动 zookeeper，我们检查三台主机的节点情况，可以看到三台机器成功启动 zookeeper，并且成功选举出一个 leader（centos7-071）。

```
[root@centos7-070 bin]# /root/software/zookeeper/bin/zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /root/software/zookeeper/bin/../conf/zoo.cfg
Client port found: 2182. Client address: 10.46.1.70. Client SSL: false.
Mode: follower
[root@centos7-071 ~]# /root/software/zookeeper/bin/zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /root/software/zookeeper/bin/../conf/zoo.cfg
Client port found: 2182. Client address: 10.46.1.71. Client SSL: false.
Mode: leader
[root@centos7-072 ~]# /root/software/zookeeper/bin/zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /root/software/zookeeper/bin/../conf/zoo.cfg
Client port found: 2182. Client address: 10.46.1.72. Client SSL: false.
Mode: follower
```

图 89 启动 zookeeper 后节点的选举情况

添加 Flink 的 HA 配置，在 centos7-070 的 flink-conf.yaml 文件中添加 HA 相关配置，主要配置的内容是修改 state.backend 为 filesystem 作为快照存储，启用检查点，可以将快照保存到 HDFS；存储 JobManager 的元数据到 HDFS；使用 zookeeper 搭建高可用；zookeeper 的地址 high-availability.zookeeper.quorum: centos7-070:2181,centos7-071:2181,centos7-072:2181。将配置好 HA 的 flink-conf.yaml 分发到另外两个节点。

到 centos7-071 修改 flink-conf.yaml 中的配置，将 JobManager 设置为自己节点的名称：jobmanager.rpc.address: centos7-071。在各节点上修改 masters 文件，添加双主节点，分别添加 centos7-070:8081, centos7-071:8081。

启动 zookeeper 集群、HDFS 集群，最后启动 Flink 集群，测试 HA 的效果通过 Web 分别查看两个 master 节点的状态，尝试 kill 掉一个节点并通过 Web 查看另外的一个节点。我们首先通过 ssh 远程

连接 centos7-070 和 centos7-071，端口分别为 9000 和 9001，在对 9000 端口进行访问的时候，网页是打不开的，因为这时候 centos7-071 是 leader，那我们打开 9001 端口是可以访问的，符合我们的预期，如图 90 所示。

Job Manager	
Configuration	Logs Stdout
Key	Value
high-availability	zookeeper
high-availability.storageDir	hdfs://flink/ha/
high-availability.zookeeper.quorum	centos7-070:2182,centos7-071:2182,centos7-072:2182
jobmanager.heap.size	1024m
jobmanager.rpc.address	centos7-071
jobmanager.rpc.port	33259
parallelism.default	1
rest.port	8081
state.backends.fs.checkpointdir	hdfs://flink-checkpoints
state.backend	filesystem
taskmanager.heap.size	1024m
taskmanager.numberOfTaskSlots	2
web.tmpdir	/tmp/flink-web-77650df2-b1af-47ff-b169-858c2bcf710c

图 90 访问 9001 端口 Web 页面

在对 Centos7-071 的 jobmanager 进程进行 kill 中断后，原来的 standby 节点 centos7-070 的 webui 界面成功访问，而 Centos7-071 的 webui 界面无法访问，完成了主备切换，如图 91 所示，而此时的进程情况如图 92 所示，可以看到代表 standalone 的 jobmanager 进程在 centos7-071 上被杀死。

Job Manager	
Configuration	Logs Stdout
Key	Value
high-availability	zookeeper
high-availability.storageDir	hdfs:///flink/ha/
high-availability.zookeeper.quorum	centos7-070:2182,centos7-071:2182,centos7-072:2182
jobmanager.heap.size	1024m
jobmanager.rpc.address	centos7-070
jobmanager.rpc.port	33367
parallelism.default	1
rest.port	8081
state.backends.fs.checkpointdir	hdfs:///flink-checkpoints
state.backend	filesystem
taskmanager.heap.size	1024m
taskmanager.numberOfTaskSlots	2
web.tmpdir	/tmp/flink-web-3f143c81-53ec-45cd-83f0-94be4c4adde4

图 91 访问 9000 端口 Web 页面

```

[root@centos7-071 flink]# jps
75312 QuorumPeerMain
100215 Jps
78104 TaskManagerRunner
75407 DataNode

[root@centos7-070 conf]# jps
1104816 StandaloneSessionClusterEntrypoint
570040 WrapperSimpleApp
1105298 TaskManagerRunner
1109206 Jps
1100455 QuorumPeerMain
869940 JobHistoryServer
1101039 SecondaryNameNode
1100783 NameNode

```

图 92 两节点的进程情况

而这里让我非常困惑的点是，当我查看 071 和 070 两个节点的 zk 的状态时，071 还是 leader，070 还是 follower，这与我原先的预期是有点不符的，按理来说不应该要重新选举了么？但其实这里有两套 leader 的体系，现在重新选举的是 jobmanager 的 leader 而非 zk 的 leader，zk 的 leader 只有当节点本身出现错误的时候会重新选举，比如 leader 节点宕机等等。

向 HDFS 集群中上传数据，然后提交一个测试任务。如图 93。

```

[root@centos7-070 flink]# ./bin/flink run examples/batch/WordCount.jar --input hdfs:///test/input/README.txt --output hdfs:///test/output2/result.txt
Starting execution of program
Program execution finished
Job with JobID 12fb0734a185f13dc034c57144f07b0e has finished.
Job Runtime: 1006 ms
[root@centos7-070 flink]# hdfs dfs -cat /test/output2/result.txt
1 1
13 1
5d002 1
740 1
about 1
account 1
administration 1

```

图 93 测试任务的执行结果

从 web 界面中可以看出，任务成功执行且无异常；并且该任务只有一个并行度，整个 pipeline 都在单个 TaskManager slot 上执行，并且任务的耗时主要集中在 DataSource/FlatMap 阶段，体现了典型的小数据任务的特征：初始化时间远大于真正处理时间。

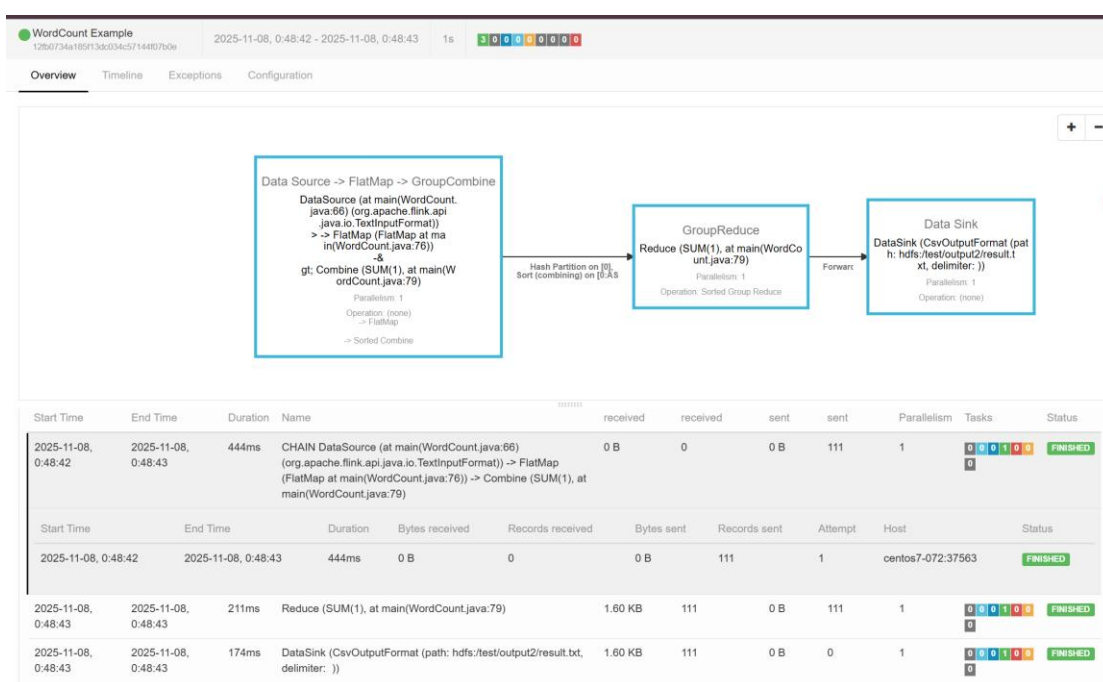


图 94 Web 页面的任务执行情况

4.4 YARN 模式安装部署 Flink

修改 centos7-070 上 Hadoop 的 yarn-site.xml，与 Spark 的 YARN 模式配置信息完全一致，参考前文；将 centos7-070 上 yarn-site.xml 文件分发到其他节点，而后启动 HDFS&YARN 集群。

将 Flink 作业提交到 Yarn 集群，对于会话方式：我们先在 flink 目录下启动 yarn-session。

```

2025-11-08 07:42:40,192 INFO org.apache.flink.shaded.curator.org.apache.curator.framework.state.ConnectionStateManager
- State change: CONNECTED
2025-11-08 07:42:40,353 INFO org.apache.flink.runtime.rest.RestClient - Rest client endpoint started.
Flink JobManager is now running on centos7-072.cumtcs.net:45485 with leader id 1043bd96-a6ba-4bc1-9cbd-8a5f1ad2322d.
JobManager Web Interface: http://centos7-072.cumtcs.net:40947
2025-11-08 07:42:40,508 INFO org.apache.flink.yarn.cli.FlinkYarnSessionCli - The Flink YARN client has been started in detached mode. In order to stop Flink on YARN, use the following command or a YARN web interface to stop it:
yarn application -kill application_1762587241316_0003

```

图 95 启动 yarn-session

而后使用 `flink run` 提交任务，并使用 YARN Web 界面查看集群情况，可以从图 96 中看到，任务 003 正在执行。最后的任务结果正如图 97 中所示，任务执行成功。

Cluster Metrics																			
Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total	Memory Reserved	Vcores Used	Vcores Total										
2	0	1	1	1	1 GB	16 GB	0 B	1	16										
Cluster Nodes Metrics																			
Active Nodes	Decommissioning Nodes			Decommissioned Nodes			Lost Nodes		Unhealthy Nodes		Rebooted Nodes								
2	0			0			0		0		0								
Scheduler Metrics																			
Scheduler Type	Scheduling Resource Type								Minimum Allocation	Maximum Allocation	Maximum Clusters								
Capacity Scheduler	[-name=memory-mb default-unit=Mi type=COUNTABLE; -name=vcores default-unit=type=COUNTABLE]								<memory:1024, vCores:1>	<memory:8192, vCores:4>	0								
Show 20 entries																			
ID	User	Name	Application Type	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU Vcores	Allocated Memory MB	Reserved CPU Vcores	Reserved Memory MB	% of Queue	% of Cluster	Progress	Tracked
application_1762587241316_0003	root	Flink session cluster	Apache Flink	default	0	Sat Nov 8 15:42:35 +0800 2025	Sat Nov 8 15:42:36 +0800 2025	N/A	RUNNING	UNDEFINED	1	1	1024	0	0	6.3	6.3		App

图 96 Web 任务正在执行

```

(a,5)
(action,1)
(after,1)
(against,1)
(all,2)

```

图 97 任务执行完毕的结果

对于分离模式：使用 `Flink run` 提交任务给 YARN 集群，检查 web 界面，可以看到任务 0005 已经运行成功。

application_1762587241316_0005	root	Flink session cluster	Apache Flink	default	0	Sat Nov 8 15:50:52 +0800 2025	Sat Nov 8 15:50:53 +0800 2025	Sat Nov 8 15:51:06 +0800 2025	FINISHED	SUCCEEDED	N/A	N/A	N/A	N/A	N/A	0.0	0.0
--------------------------------	------	-----------------------	--------------	---------	---	-------------------------------	-------------------------------	-------------------------------	----------	-----------	-----	-----	-----	-----	-----	-----	-----

图 98 分离模式下 Web 页面中任务执行完毕

4.5 Flink 应用程序开发

将 maven 项目导入 IDEA，配置 pom 依赖，导入 `flink-java`，`flink-streaming-java`，版本均为 1.10.0。

编写 WordCount 程序，执行 WordCount 程序，在 IDEA 环境内部运行，在 Windows 启动 ncat 监听 9555 端口: `ncat -lk 9555`，然后在 IDEA 中开启流处理任务，然后在 ncat 窗口输入文本，可以看到 IDEA 中成功捕捉到了文本并且进行了单词计数。如图 99。

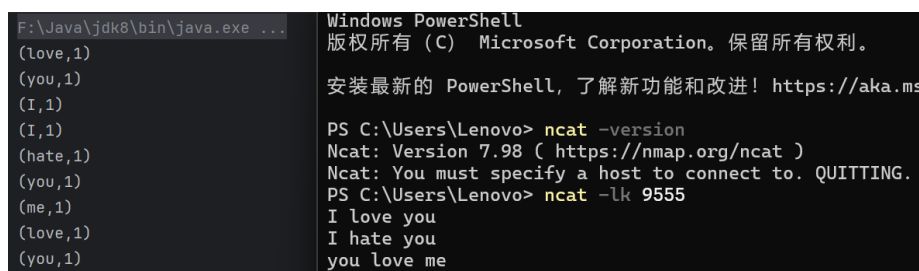


图 99 在 IDEA 中完成测试

打包程序提交集群运行: `mvn clean package`，通过 `flink run` 提交集群，这里提交到一个 Local 模式的 flink 集群，本地启动 ncat 监听 9555 端口，输入一些文本如图 100。

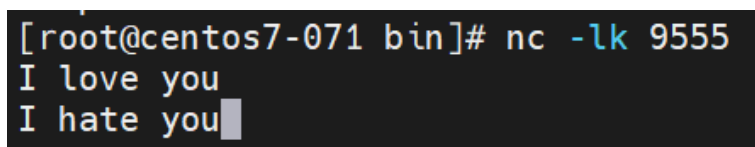


图 100 在集群中完成测试

而后我们查看 web 界面中的结果，可以看到成功执行了单词计数的任务，如图 101。

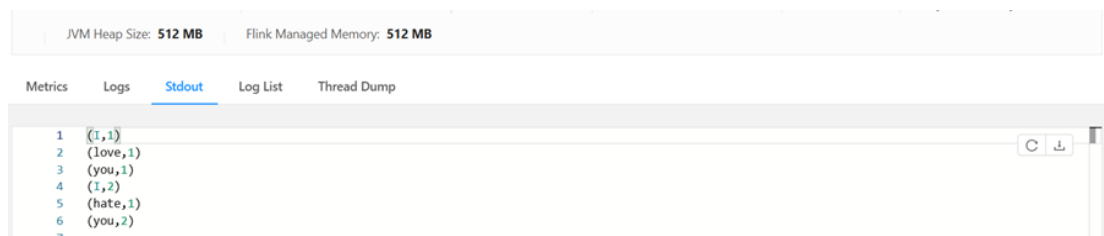


图 101 Web 页面中单词计数结果

5. 结论与讨论

通过实验四的 Flink 安装部署与开发实践，在实验过程中，我遇到了配置文件路径设置不当、ZooKeeper 与 Flink 的 Leader 选举逻辑混淆、YARN 资源调度与内存限制导致任务失败等典型问题。通过逐一排查与调试，我不仅熟悉了 Flink 集群的部署结构与关键配置文件的作用，还理解了 ZooKeeper 在分布式协调中的角色与 Flink JobManager 高可用切换的实际机制。我在 IDEA 本地与集群环境中完成测试与部署，体会到 Flink 在实时计算与状态管理方面的强大能力。通过动手搭建与调试，我不仅掌握了 Flink 的核心概念与操作方法，更提升了在分布式环境下定位问题、优化配置的工程能力，为未来从事实时数据处理与流式计算开发奠定了坚实基础。